

Thyltech



Dossier de conception Lot 3

Projet Ticketack

VALIDATION			
Nom - Fonction	Entreprise	Date	Signature
Ludovic Isaert - CEO	ID Ingénierie		

MODIFICATION			
Version	Date	Modifié par	Modification
1	18/12/2025	Léonore	Création

Table des Matières:

1. Présentation du lot 3	3
1.1. Objet du document	3
1.2. Analyse de l'existant et intégration (héritage lot 1 & 2)	3
1.3. Objectifs fonctionnels et techniques détaillés	4
1.4. Cas d'utilisation	5
2. Architecture technique détaillée	6
2.1. Vue d'ensemble des composants et flux de données	6
2.2. Choix stratégie du serveur d'inférence : vLLM vs Ollama	7
2.3. Pipeline d'orchestration RAG (workflow événementiel)	9
3. Benchmark et sélection des modèles fondationnels LLM	12
3.1. Critères de sélection et pondération	12
3.2. Candidat "local dév" (optimisé pour 16GB RAM)	12
3.3. Candidat "serveur production" (robuste & performant)	14
4. Stratégie d'indépendance et d'abstraction	16
4.1. Pattern d'architecture : abstraction par configuration	16
4.2. Contrat de données de sortie	19
5. Design du système de feedback utilisateur	20
5.1. Expérience utilisateur (UX) fluide et engageante	20
5.2. Modèle de données (schéma MySQL étendu)	22
5.3. Exploitation des données	23
6. Détails d'implémentation et sécurité	23
6.1. Structure du prompt système (template optimisé)	23
6.2. Sécurité des données et conformité	24
7. Glossaire	25

1. Présentation du lot 3

1.1. Objet du document

Le présent dossier a pour objectif de décrire la conception fonctionnelle et technique du Lot 3 du projet Ticketack.

Ce lot porte sur l'intégration d'une intelligence artificielle d'assistance aux solveurs (1), s'appuyant sur un mécanisme de RAG – Retrieval Augmented Generation (2) et sur la base de connaissances mise en place précédemment.

Contrairement à une simple consultation de solutions existantes, cette IA analysera automatiquement le contenu des tickets (titre, description, pièces jointes) afin de proposer des pistes de résolution contextualisées (3), issues de tickets similaires, de documents internes ou de procédures référencées.

Une fois les suggestions générées, le solveur (4) pourra les accepter ou les refuser, fournir un retour explicatif et ainsi contribuer à une amélioration continue du modèle, tout en bénéficiant d'outils de supervision et de suivi des performances de l'IA.

1.2. Analyse de l'existant et intégration (héritage lot 1 & 2)

Pour concevoir l'architecture du Lot 3, il est impératif d'analyser finement l'héritage technique des lots précédents, qui servent de substrat au déploiement de l'IA. Une mauvaise intégration avec ces couches existantes constituerait une dette technique immédiate.

L'Architecture Applicative (Lot 1) repose sur un monolithe modulaire développé en **Laravel 11**, couplé à une interface utilisateur réactive en **React**, servie via le protocole **Inertia.js**. Ce choix architectural, validé lors des phases précédentes, garantit une fluidité de navigation de type SPA (*Single Page Application*) tout en conservant la robustesse et la sécurité du backend PHP. La persistance des données métier (utilisateurs, rôles, tickets, équipements) est assurée par une base de données relationnelle **MySQL**. L'authentification et la gestion des permissions, basées sur les standards de Laravel, sont déjà en place et devront être étendues pour sécuriser les interactions avec l'IA.

L'Architecture Data (Lot 2) a introduit une complexité nécessaire pour gérer les données non structurées. Le stockage des fichiers (pièces jointes, logs, captures d'écran) est assuré par un *Data Lakehouse* local basé sur **MinIO**, offrant une compatibilité S3. L'indexation vectorielle, cœur du moteur de recherche sémantique, est gérée par **LanceDB**, une base vectorielle *serverless* et embarquée qui stocke ses données directement sur le stockage objet, offrant un découplage fort entre le calcul et le stockage. Un pipeline ETL (*Extract, Transform, Load*) asynchrone, développé en **Python** et orchestré par des files d'attente, assure déjà l'extraction (OCR), le découpage (*chunking*) et l'embedding des documents via le modèle **BGE-M3**.

La conception du Lot 3 doit s'insérer dans ce flux sans créer de goulot d'étranglement bloquant. L'application Laravel, qui gère les interactions utilisateur en temps réel, ne doit jamais attendre une réponse synchrone de l'IA, dont le temps de génération peut varier de quelques secondes à plusieurs dizaines de secondes selon la charge et la complexité de la requête. L'architecture doit donc être événementielle, asynchrone et résiliente par conception.

1.3. Objectifs fonctionnels et techniques détaillés

Le périmètre du Lot 3 se précise techniquement autour de quatre axes majeurs :

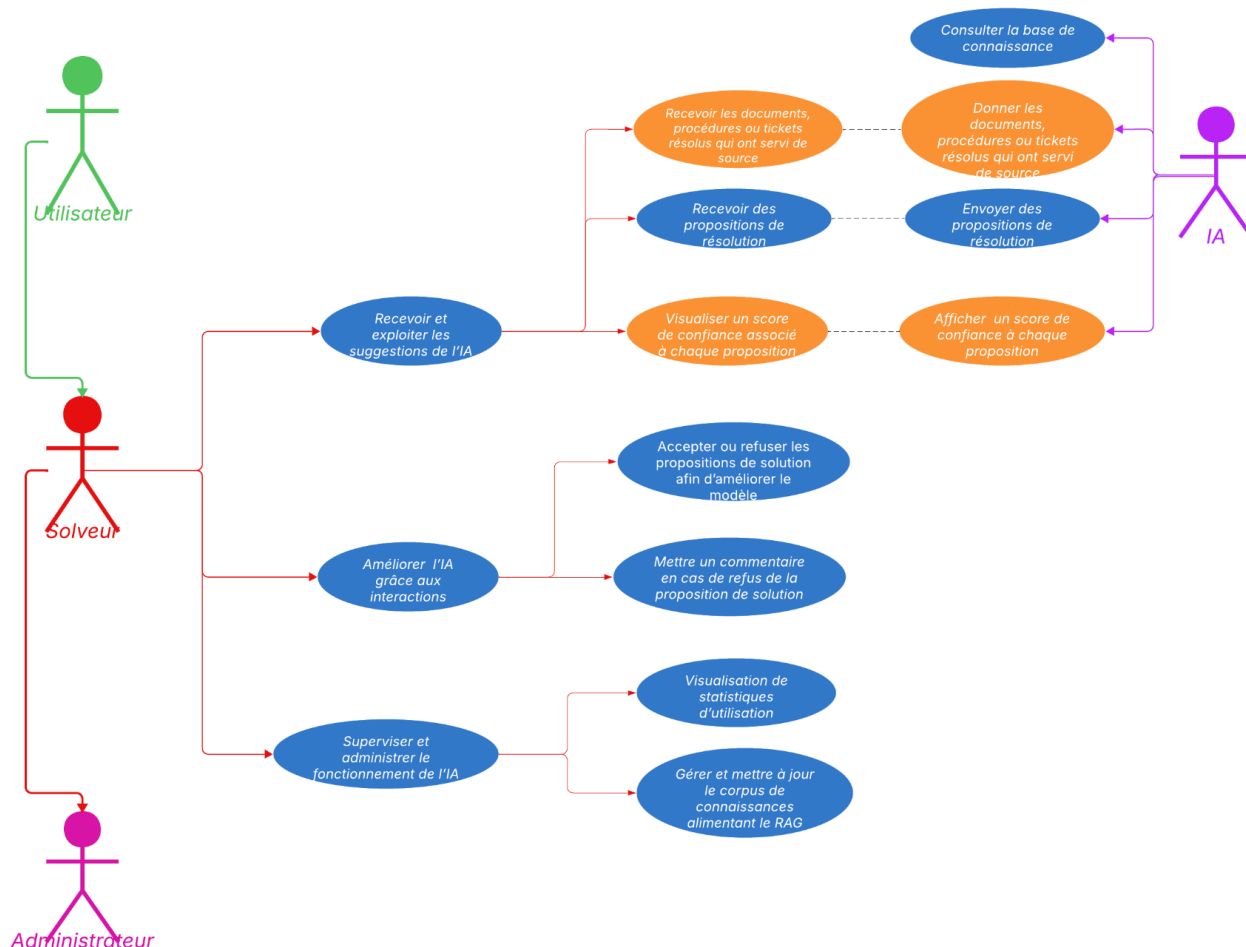
1. **Génération Automatisée de Pistes de Résolution** : Dès la création d'un ticket, le système doit déclencher une analyse autonome. Il doit comprendre le contexte implicite de la demande, interroger la base vectorielle pour récupérer les tickets passés similaires et les procédures techniques pertinentes (PDF, Docs), et rédiger une proposition de solution structurée. Cette proposition ne doit pas être une simple concaténation d'extraits, mais une synthèse raisonnée.
2. **Système de Feedback et Apprentissage Humain** : L'IA n'est pas infaillible. Le solveur doit conserver un rôle central de validation. Il doit pouvoir accepter, modifier ou rejeter la suggestion de l'IA. Ces interactions ne sont pas de simples actions d'interface ; elles constituent la donnée d'entraînement pour les futures itérations du modèle. Le design de ce système de feedback est critique pour créer une boucle d'amélioration continue (*Data Flywheel*).
3. **Stratégie d'Indépendance Technologique** : Le domaine de l'IA évoluant à une vitesse fulgurante, le système doit être conçu pour ne pas dépendre d'un modèle unique. Il doit pouvoir basculer de manière transparente entre différents LLM (par exemple, utiliser Mistral pour sa maîtrise du français ou Qwen pour ses capacités de raisonnement logique) sans nécessiter de refonte du code applicatif. Cette exigence impose une couche d'abstraction logicielle rigoureuse.
4. **Déploiement Hybride et Scalabilité** : Le projet doit concilier deux réalités matérielles : une configuration "légère" pour les postes de développement (typiquement des machines avec 16GB de RAM unifiée) et une configuration "robuste" pour le serveur de production (équipé de GPU avec VRAM > 24GB). Le choix des modèles et des moteurs d'inférence doit refléter cette dualité sans rompre la compatibilité fonctionnelle.

1.4. Cas d'utilisation

Actuel V3

Théo Colin | Décembre 9, 2025

Administrateur
hérite de
Solveur, qui
hérite de
Utilisateur



Le lot 3 de l'application est destiné à être utilisé exclusivement par les Solveurs et Administrateurs. Son objectif est d'intégrer une intelligence artificielle d'assistance capable d'analyser automatiquement le contenu des tickets afin de proposer des pistes de résolution pertinentes et contextualisées, en s'appuyant sur la base de connaissances constituée lors du lot précédent.

Ce lot de fonctionnalités apporte aux Solveurs un accompagnement intelligent dans le traitement des tickets, en leur fournissant des suggestions de solutions issues de tickets similaires, de documents internes ou de procédures existantes. Les Solveurs conservent un rôle central dans la validation des propositions, en pouvant accepter ou refuser les suggestions et formuler des retours explicatifs.

Ainsi, le lot 3 permet aux Solveurs novices de monter en compétence plus rapidement grâce à des recommandations guidées, tout en offrant aux Solveurs expérimentés un gain d'efficacité sur les tickets récurrents ou complexes. Cette assistance contribue à une réduction du temps moyen de résolution (MTTR) et à une amélioration continue de la qualité du support, grâce à la boucle d'apprentissage alimentée par les interactions humaines.

Enfin, ce lot marque l'aboutissement de la chaîne fonctionnelle amorcée par les lots précédents, en transformant la base de connaissances et la recherche sémantique en un véritable moteur décisionnel intelligent, au cœur du système Ticketack.

2. Architecture technique détaillée

L'architecture du Lot 3 ne se contente pas d'ajouter des modules ; elle transforme la topologie du système. Nous passons d'une architecture web classique à une architecture distribuée orientée **Agents et Services d'Inférence**. Cette évolution est nécessaire pour supporter la charge de calcul des LLM et garantir la réactivité de l'interface utilisateur.

2.1. Vue d'ensemble des composants et flux de données

Le système s'articule désormais autour de quatre pôles majeurs, communiquant de manière asynchrone. Cette séparation des responsabilités est cruciale pour la maintenabilité et la scalabilité.

1. **Le Producteur d'Événements (Laravel - Backend Applicatif)** : Il conserve son rôle de chef d'orchestre des processus métier. Sa responsabilité dans le Lot 3 est de détecter les événements déclencheurs (création d'un ticket, ajout d'un commentaire critique) et d'émettre un signal standardisé. Il ne traite aucune logique d'IA directement.
2. **L'Orchestrateur IA (Service Python/Celery)** : C'est le nouveau "cerveau" introduit par le Lot 3. Il s'agit d'un service autonome, découplé du monolithe PHP. Il reçoit les signaux via Redis, orchestre la récupération de contexte (RAG) en interrogeant LanceDB, construit le *prompt* système, interroge le serveur d'inférence, et post-traite la réponse (parsing JSON, validation). Il utilise **Celery** pour la gestion des files d'attente de tâches, garantissant que les demandes de génération ne sont jamais perdues, même en cas de surcharge temporaire.
3. **Le Serveur d'Inférence (Moteur LLM)** : Ce composant est dédié exclusivement à l'exécution mathématique des modèles de langage. Il charge les poids du modèle (plusieurs gigaoctets) en VRAM et expose une API standardisée (compatible OpenAI). Son rôle est de transformer une suite de tokens (le prompt) en une suite de tokens (la réponse) le plus rapidement possible. La distinction entre l'orchestrateur (logique) et le serveur d'inférence (calcul) permet de scaler ces deux couches indépendamment.
4. **Le Stockage de Feedback et Métadonnées (MySQL & LanceDB)** : Le schéma relationnel MySQL est étendu pour stocker les suggestions générées, leurs métadonnées (modèle utilisé, temps de génération, score de confiance) et les feedbacks utilisateurs. Parallèlement, LanceDB continue de servir le contexte

vectorel, mais pourra à terme ingérer les "meilleures réponses" validées pour enrichir la base de connaissances.

2.2. Choix stratégie du serveur d'inférence : vLLM vs Ollama

Le choix du moteur d'exécution des modèles est l'une des décisions les plus critiques pour la performance en production. Deux solutions dominent actuellement l'écosystème open-source : **Ollama** et **vLLM**. Une analyse comparative rigoureuse est nécessaire pour justifier le choix architectura

Critère	Ollama	vLLM	Implications pour Ticketack
Philosophie & cible	Conçu pour la simplicité, le développement local et les machines grand public (Mac M-series, GPU gamer).	Conçu pour la production haute performance en datacenter. Optimisé pour maximiser l'utilisation du GPU et le débit.	Ollama est idéal pour les dev, vLLM pour le serveur.
Performances (débit)	Performant pour un utilisateur unique. Cependant, l'architecture ne gère pas efficacement la concurrence : les requêtes sont souvent mises en file d'attente séquentielle, effondrant le débit sous charge.	Excellent. Utilise des techniques avancées comme PagedAttention (gestion mémoire non contiguë) et Continuous Batching (traitement simultané de requêtes à différents stades). Peut être jusqu'à 3.2x plus rapide qu'Ollama en charge.	vLLM est impératif pour supporter plusieurs solveurs simultanés sans latence excessive.
Gestion de la latence	La latence augmente linéairement avec le nombre de requêtes simultanées.	La latence par token reste stable même avec plusieurs requêtes simultanées, grâce à l'optimisation des noyaux CUDA.	vLLM garantit une expérience utilisateur fluide (UX).
Gestion mémoire (VRAM)	Flexible. Charge et décharge les modèles à la volée, ce qui est pratique en dev mais coûteux en temps lors des changements de contexte.	Pré-alloue la VRAM pour le cache KV (Key-Value) afin d'éviter la fragmentation. Plus rigide mais beaucoup plus efficace pour les contextes longs (RAG).	vLLM optimise l'investissement matériel du serveur.
Compatibilité API	Propose sa propre API REST et une compatibilité OpenAI partielle (proxy).	Compatibilité OpenAI native quasi-totale, facilitant l'intégration avec des bibliothèques standards.	Égalité relative, mais vLLM simplifie le standard.

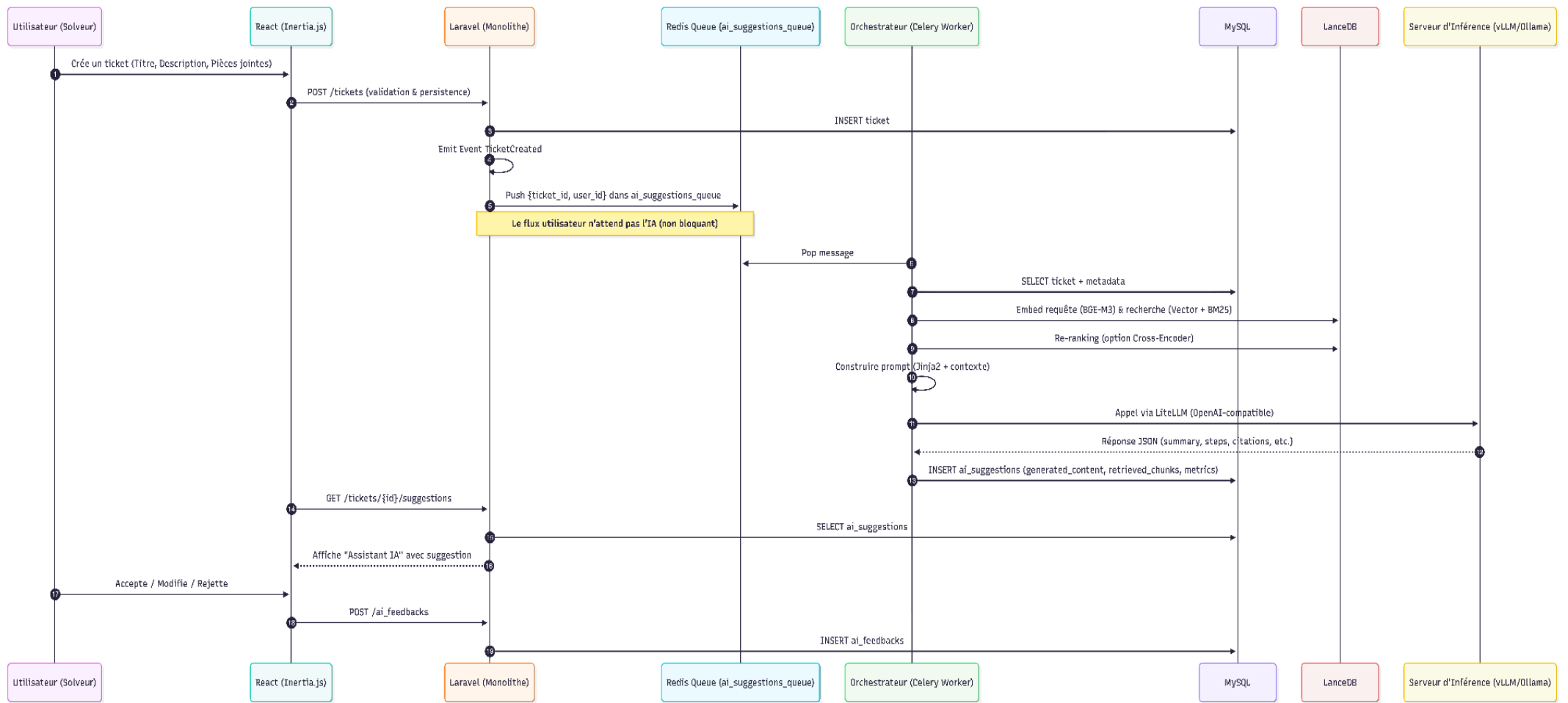
Décision Architecturale : Une Stratégie Hybride Nous refusons de faire un compromis unique qui pénaliserait soit les développeurs, soit les utilisateurs finaux. Nous adoptons une stratégie différenciée :

- **Environnement de Développement (Local)** : Utilisation d'**Ollama**. Sa simplicité d'installation (binaire unique) et sa faible empreinte mémoire permettent aux développeurs de travailler sur des machines standard (MacBook M1/M2/M3 ou PC avec GPU type RTX 3060/4060) sans configuration complexe de drivers CUDA. Cela réduit la barrière à l'entrée pour les nouveaux développeurs rejoignant le projet.
- **Environnement de Production (Serveur)** : Utilisation de **vLLM**. Pour garantir un temps de réponse acceptable lorsque plusieurs solveurs travaillent simultanément sur des tickets complexes, les fonctionnalités de *batching* continu de vLLM sont indispensables. vLLM sera déployé dans un conteneur Docker dédié, avec une configuration stricte de l'allocation VRAM (paramètre `--gpu-memory-utilization`) pour cohabiter avec les autres services si nécessaire.

2.3. Pipeline d'orchestration RAG (workflow évènementiel)

L'orchestrateur ne doit pas être un simple script monolithique. Il est conçu comme une application Python modulaire :

Séquence technique d'une suggestion IA :



1. Détection et Émission (Laravel) :

- L'utilisateur crée un ticket via l'interface React.
- Laravel valide et persiste le ticket dans MySQL.
- Un *Event* Laravel `TicketCreated` est instancié.
- Un *Listener* associé à cet événement formate un payload léger (ID du ticket, User ID) et le pousse dans une file d'attente Redis dédiée : `ai_suggestions_queue`. Ce découplage assure que si le service IA est indisponible, la création du ticket n'est pas ralentie.

2. Consommation et Enrichissement (Worker Python) :

- Le worker Celery dépile le message.
- Il effectue une requête SQL pour récupérer les données textuelles complètes du ticket (Titre, Description, Catégorie, Équipement concerné).

3. Stratégie de Récupération (Retrieval) - Interaction avec le Lot 2 :

- **Encodage de la Requête** : Le worker utilise le modèle d'embedding **BGE-M3** pour transformer le texte du ticket en vecteur. Il est impératif d'utiliser exactement le même modèle que celui du Lot 2 pour garantir la cohérence géométrique de l'espace vectoriel.
- **Recherche Hybride** : Il interroge **LanceDB** pour récupérer les "chunks" les plus pertinents. Une stratégie de recherche hybride est appliquée : recherche vectorielle (pour le sens) pondérée par une recherche par mots-clés (BM25) pour capturer les codes d'erreur spécifiques ou les noms de machines exacts.
- **Re-ranking** : Si la performance brute le permet, une étape de *re-ranking* (avec un modèle Cross-Encoder léger) est appliquée sur les 20 résultats bruts pour ne garder que les 5 plus pertinents, augmentant la précision du contexte fourni au LLM.

4. Ingénierie du Prompt (Prompt Construction) :

- Le worker charge le template de prompt adapté (voir section 6.1).
- Il injecte le contexte récupéré (les *chunks* de documentation et tickets résolus).
- Il injecte la demande courante.
- Il ajoute des instructions strictes de formatage (JSON Schema) pour garantir que la sortie soit exploitable informatiquement.

5. Inférence (Génération) :

- Le worker transmet le prompt complet à l'API du serveur d'inférence (vLLM en prod, Ollama en dev) via la librairie d'abstraction **LiteLLM**.
- La génération est configurée avec une température faible (0.1 - 0.3) pour favoriser la factualité et réduire les hallucinations créatives.

6. Post-traitement et Livraison :

- Le worker reçoit la réponse brute.
- Il tente de parser le JSON. En cas d'erreur de syntaxe (JSON malformé), une logique de *retry* peut être déclenchée ou une erreur est loguée.
- Il sauvegarde la suggestion structurée dans la table `ai_suggestions` de la base MySQL.

3. Benchmark et sélection des modèles fondationnels LLM

La sélection du modèle de langage est un exercice d'équilibre complexe. Il ne s'agit pas de trouver le "meilleur" modèle dans l'absolu, mais celui qui offre le meilleur compromis entre **intelligence** (raisonnement), **contexte** (capacité RAG), **maîtrise du français**, et **coût computationnel** (VRAM).

3.1. Critères de sélection et pondération

Pour le projet Ticketack, les critères sont hiérarchisés comme suit :

1. **Support du Français (Critique)** : Le modèle doit saisir les subtilités techniques et idiomatiques du français, langue de travail des solveurs.
2. **Performance RAG et Fenêtre de Contexte (Critique)** : Le modèle doit pouvoir ingérer un contexte dense (plusieurs milliers de tokens de documentation) sans "oublier" d'informations et sans halluciner. Une fenêtre de 8k est un minimum absolu ; 32k ou 128k est idéal.
3. **Licence (Majeur)** : Une licence **Apache 2.0** ou **MIT** est privilégiée pour garantir une liberté d'utilisation commerciale totale et éviter les contraintes légales futures des licences "Community" restrictives (comme celles de Llama pour les très gros acteurs, bien que moins gênant ici, ou les licences restrictives de certains modèles Qwen passés).
4. **Contraintes Matérielles (Hardware)** :
 - a. *Dev Local* : Doit fonctionner de manière fluide sur une machine avec 16GB de RAM (impliquant une empreinte mémoire < 12GB pour laisser l'OS respirer).
 - b. *Prod Serveur* : Doit maximiser l'utilisation d'un GPU serveur standard (ex: NVIDIA A10G 24GB ou L40S 48GB) sans nécessiter de cluster multi-GPU complexe et coûteux.

3.2. Candidat "local dev" (optimisé pour 16GB RAM)

L'objectif est de fournir aux développeurs un modèle suffisamment intelligent pour valider le pipeline technique (format JSON, prise en compte du contexte) sans mettre à genoux leur poste de travail.

Analyse comparative des modèles <14B :

Modèle	Paramètres	License	Français	Taille de contexte	VRAM (4-bit)	Verdict
Mistral-Nemo 12B	12B	Apache 2.0	Excellent (Natif)	128k	~7.5 - 8 GB	Champion. Développé conjointement par Mistral et Nvidia. Il surpasse Llama 3.1 8B en raisonnement et multilinguisme. Son tokenizer "Tekken" est particulièrement efficace pour le français, réduisant le nombre de tokens nécessaires.
Qwen 2.5 14B	14B	Apache 2.0	Excellent	128k	~9 - 10 GB	Très puissant , notamment en code et logique. Cependant, son empreinte mémoire (10GB) est dangereusement proche de la limite sur une machine 16GB si l'on fait tourner Docker (MinIO, LanceDB, Laravel) et un IDE simultanément. Risque de swap élevé.
Llama 3.2 3B	3B	Llama Community	Bon	128k	~2.5 GB	Extrêmement léger et rapide. Cependant, ses capacités de raisonnement complexe et de suivi d'instructions strictes (JSON schema complexe) sont limitées pour un cas d'usage de support technique. Utile uniquement pour des tests d'intégration très rapides.
Ministral 8B	8B	Apache 2.0	Excellent	128k	~5.5 GB	Un excellent challenger , conçu pour l'Edge. Très efficace, mais Mistral-Nemo 12B offre un surcroît d'intelligence appréciable pour le RAG complexe si la RAM le permet.

Recommandation "Local Dev" : Mistral-Nemo 12B (Instruct) quantifié en 4-bit (GGUF/AWQ). C'est le compromis optimal. Avec une quantification 4-bit, il occupe environ 8GB, laissant une marge de manœuvre suffisante sur une machine de 16GB. Sa fenêtre de contexte de 128k permet de tester le RAG sans tronquer arbitrairement les documents, simulant fidèlement le comportement de production.

3.3. Candidat "serveur production" (robuste & performant)

Pour la production, nous visons la qualité maximale de réponse et la robustesse du suivi d'instructions, en supposant un serveur équipé d'un GPU de 24GB VRAM (le standard économique du cloud actuel, type AWS g5.2xlarge).

Analyse comparative des modèles 20B - 70B :

Modèle	Paramètres	License	Français	Taille de contexte	VRAM (4-bit)	Verdict
Mistral Small 3 (24B)	24B	Apache 2.0	SOTA (État de l'art)	32k	~14 - 16 GB	Recommandé. Sorti début 2025 , ce modèle est une révolution pour l'auto-hébergement. Il est "knowledge-dense" , bat GPT-4o-mini sur les benchmarks, et surtout, il est conçu pour tenir confortablement sur un GPU 24GB. Sa licence est permissive.
Qwen 2.5 32B	32B	Apache 2.0	Très Fort	128k	~19 - 22 GB	Excellent en mathématiques et code. Cependant, en 4-bit, il sature presque entièrement les 24GB de VRAM . Cela laisse très peu de place pour le cache KV (le contexte de la conversation/RAG), ce qui limiterait drastiquement la concurrence ou la longueur des documents analysables.
Llama 3.3 70B	70B	Llama Community	Bon	128k	~40 GB	Nécessite un matériel beaucoup plus onéreux (2x 24GB ou 1x A100 40GB/80GB). Le rapport coût/performance n'est pas justifié pour Ticketack par rapport à Mistral Small 3 qui offre des performances similaires pour un coût d'infrastructure divisé par deux.

Recommandation "Serveur Production" : Mistral Small 3 (24B) Instruct (v2501). Ce modèle représente le "sweet spot" architectural de 2025.

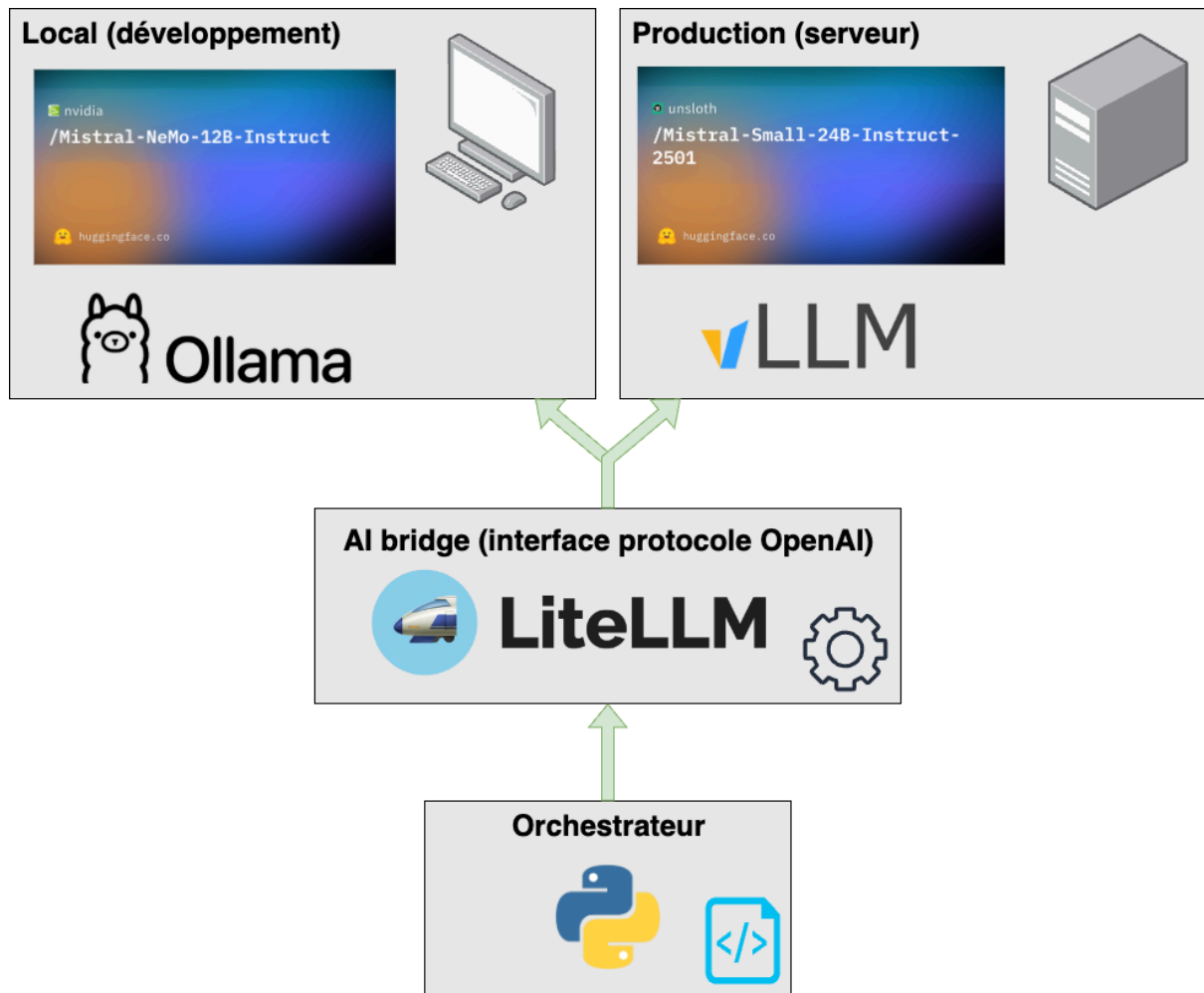
- **Adéquation Matérielle** : En quantification 4-bit (AWQ ou GPTQ), il pèse environ 14-15GB. Sur un GPU de 24GB, cela laisse environ 9-10GB de VRAM libre pour le cache KV de vLLM. Cela permet de gérer de larges lots de requêtes simultanées ou de longs contextes RAG sans erreur OOM (*Out Of Memory*).
- **Performance** : Ses benchmarks le placent au niveau des modèles 70B de la génération précédente, avec une latence bien moindre.
- **Configuration de Déploiement** : Il sera servi par **vLLM** avec une quantification AWQ 4-bit. Si le budget permet un GPU A100 (40GB/80GB), on pourra passer en FP16 pour une précision maximale, mais le gain qualitatif sera marginal pour du support technique comparé au coût.

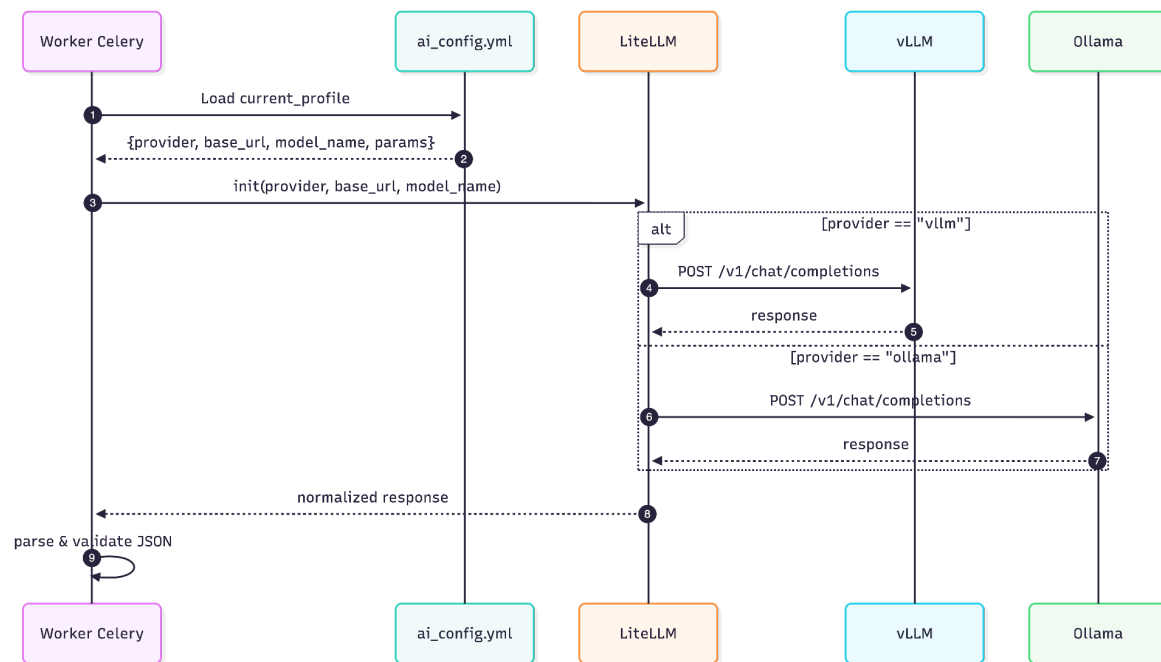
4. Stratégie d'indépendance et d'abstraction

L'écosystème de l'IA est instable : un modèle dominant aujourd'hui peut être obsolète dans six mois. Lier le code métier à l'API spécifique d'un modèle ou d'un fournisseur serait une erreur stratégique majeure. Nous mettons en place une couche d'abstraction stricte.

4.1. Pattern d'architecture : abstraction par configuration

L'application ne doit contenir aucune référence "en dur" à Mistral, Qwen ou OpenAI.





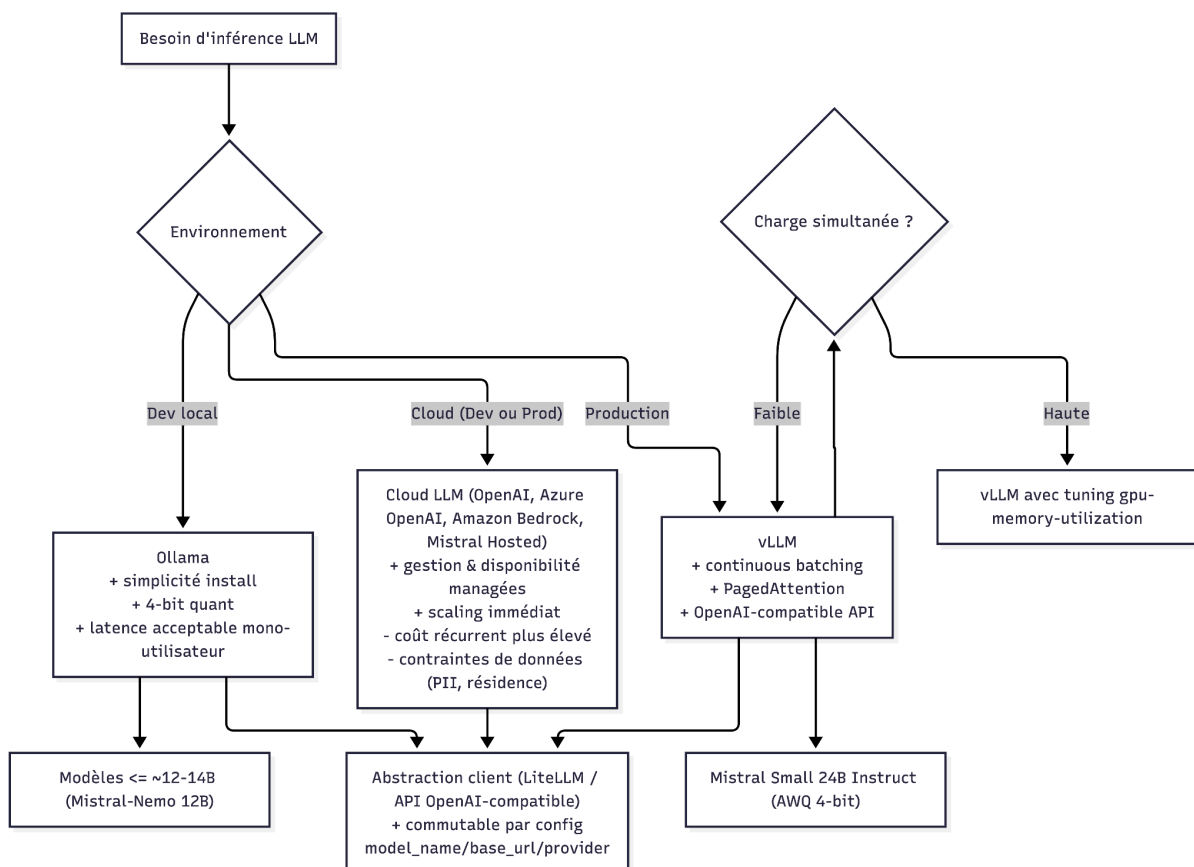
1. **Configuration Centralisée** : Un fichier de configuration (ex: `ai_config.yml`) pilote le comportement de l'IA. Il définit le fournisseur actif, les endpoints, et les hyper-paramètres. Cela permet de changer de modèle par simple redémarrage du service, sans déploiement de code.

```

current_profile: "production"
profiles:
  production:
    provider: "vllm"
    base_url: "http://ai-inference-service:8000/v1"
    model_name: "mistralai/Mistral-Small-24B-Instruct-2501"
    api_key: "${VLLM_API_KEY}"
    context_window: 32768
    timeout: 60
  development:
    provider: "ollama"
    base_url: "http://localhost:11434/v1"
    model_name: "mistral-nemo:12b-instruct-q4_K_M"
    context_window: 128000
  
```

2. **Médiateur Logiciel (LiteLLM)** : Plutôt que d'implémenter manuellement des adaptateurs pour chaque API (Ollama, vLLM, Azure, etc.), nous intégrons la librairie **LiteLLM** dans le worker Python.

- a. **Rôle** : LiteLLM agit comme un proxy universel. Il normalise les appels vers plus de 100 fournisseurs en une interface unique compatible OpenAI Chat Completion.
 - b. **Avantage** : Si demain l'entreprise utilisatrice décide de passer sur un modèle hébergé sur Azure ou AWS Bedrock, seule la configuration change. Le code Python reste identique.
3. **Gestion Dynamique des Prompts (Templating Jinja2)** : Les prompts ne sont pas des chaînes de caractères dans le code, mais des fichiers *templates* (Jinja2). Cela est crucial car chaque modèle réagit différemment à la structure du prompt (certains préfèrent des balises XML, d'autres du Markdown).
 - a. Le système charge le template `ticket_resolution.j2`.
 - b. Il injecte les variables (contexte, ticket).
 - c. Cette séparation permet aux "Prompt Engineers" d'itérer sur la qualité des réponses sans toucher au code backend.



4.2. Contrat de données de sortie

L'indépendance passe aussi par la standardisation de la sortie. L'orchestrateur ne doit pas avoir à deviner comment parser la réponse. Nous imposons un schéma JSON strict via

Pydantic. Indépendamment du modèle utilisé, l'orchestrateur valide que le JSON reçu respecte le schéma attendu (champs `summary`, `steps`, `citations`). Si le modèle échoue (ce qui arrive avec les petits modèles), une logique de "Retry" avec correction automatique (passant l'erreur de validation au modèle pour qu'il se corrige) est déclenchée.

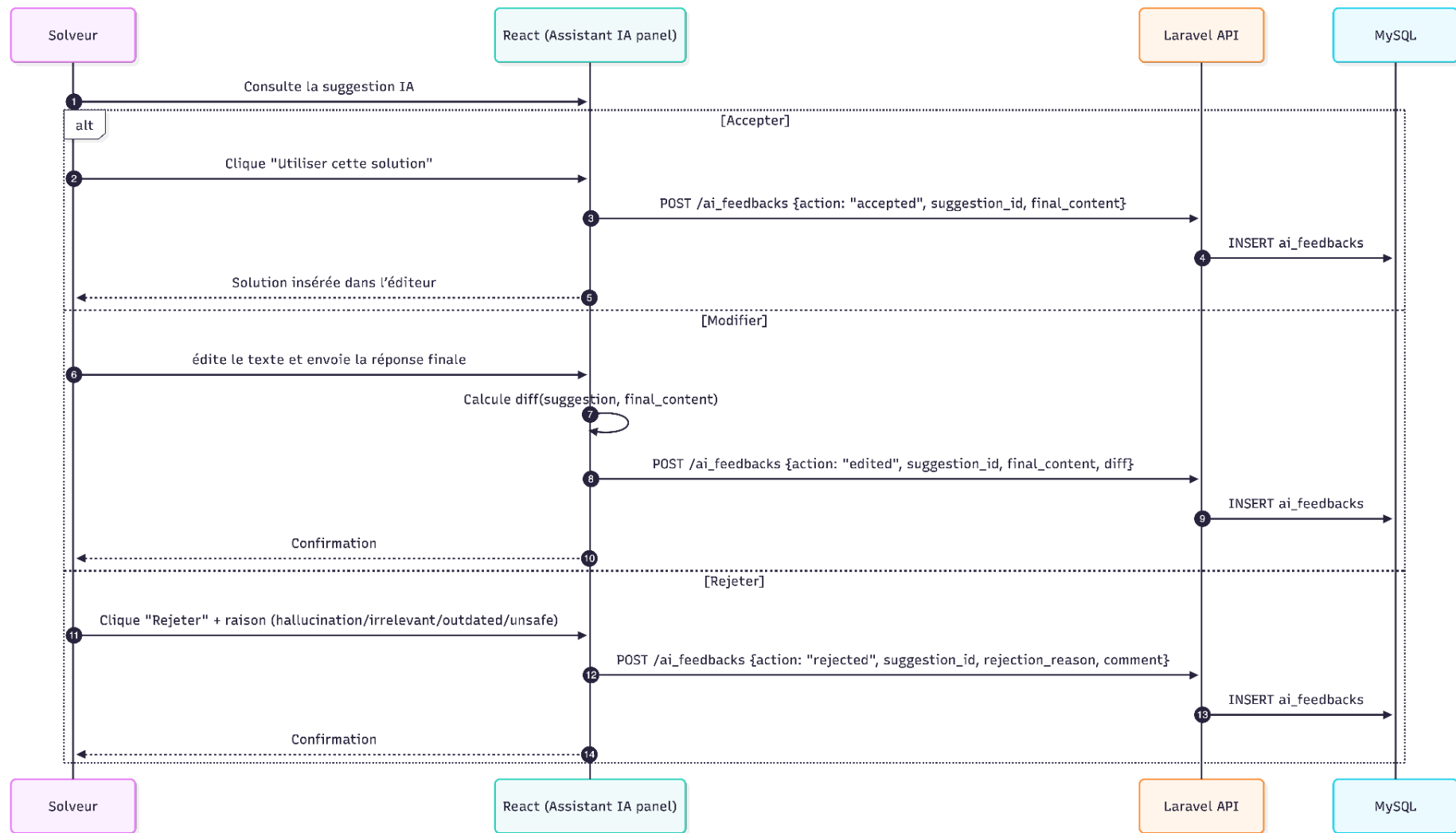
5. Design du système de feedback utilisateur

La mise en place d'un système RAG n'est pas une fin en soi. La qualité des réponses dépendra de la capacité du système à apprendre de ses erreurs. Le feedback des solveurs est la donnée la plus précieuse du système.

5.1. Expérience utilisateur (UX) fluide et engageante

L'intégration dans l'interface React (Lot 1) doit être conçue pour capter le feedback sans alourdir la charge cognitive du solveur.

1. **Présentation** : La suggestion de l'IA apparaît dans un bloc distinct (ex: accordéon ou panneau latéral) intitulé "Assistant IA". Elle ne remplit pas automatiquement le champ de réponse pour éviter les envois accidentels.
2. **Actions de Feedback Implicites et Explicites** :
 - a.  **Accepter / Insérer (Feedback Positif Fort)** : Le solveur clique sur "Utiliser cette solution". Le texte est copié dans l'éditeur. Le système enregistre que la suggestion était pertinente.
 - b.  **Modifier (Feedback Correctif - Le plus riche)** : Le solveur modifie le texte suggéré avant de l'envoyer. Le système capture le "diff" (la différence) entre la suggestion brute et la version finale envoyée au client. C'est ce delta qui servira à affiner les futurs modèles (Fine-tuning).
 - c.  **Rejeter (Feedback Négatif)** : Le solveur signale une suggestion inutile. Une modale légère demande la raison : "Hallucination" (faux faits), "Hors Sujet", "Dangereux", ou "Obsolète".



5.2. Modèle de données (schéma MySQL étendu)

Le stockage de ces interactions nécessite des tables dédiées, optimisées pour l'analyse et le ré-entraînement.

Schéma relationnel proposé :

AI_SUGGESTIONS		
bigint	id	PK
bigint	ticket_id	FK
json	model_config_snapshot	
char	prompt_hash	
json	generated_content	
json	retrieved_chunks	
float	confidence_score	
int	processing_time_ms	
timestamp	created_at	

AI_FEEDBACKS			
bigint	id	PK	
bigint	suggestion_id	FK	
bigint	user_id	FK	
enum	action_type		accepted edited rejected
text	final_content		
enum	rejection_reason		hallucination irrelevant outdated unsafe other
text	rejection_comment		
timestamp	created_at		

```
CREATE TABLE ai_suggestions (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  ticket_id BIGINT NOT NULL,
  model_config_snapshot JSON NOT NULL COMMENT 'Version du modèle et paramètres (temp, top_p) au moment du tirage',
  prompt_hash CHAR(64) NOT NULL COMMENT 'Empreinte du template de prompt utilisé (A/B testing)',
  generated_content JSON NOT NULL COMMENT 'La réponse structurée brute de l IA',
  retrieved_chunks JSON COMMENT 'IDs et scores des documents RAG utilisés pour générer cette réponse',
  confidence_score FLOAT COMMENT 'Score de confiance auto-évalué par l IA ou le système',
  processing_time_ms INT COMMENT 'Métriques de performance',
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (ticket_id) REFERENCES tickets(id)
);

CREATE TABLE ai_feedbacks (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  suggestion_id BIGINT NOT NULL,
  user_id BIGINT NOT NULL COMMENT 'Le solveur qui a donné le feedback',
  action_type ENUM('accepted', 'edited', 'rejected') NOT NULL,
  final_content TEXT COMMENT 'Contenu final réellement envoyé au client (si edited/accepted)',
  rejection_reason ENUM('hallucination', 'irrelevant', 'outdated', 'unsafe', 'other') NULL,
  rejection_comment TEXT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (suggestion_id) REFERENCES ai_suggestions(id)
);
```

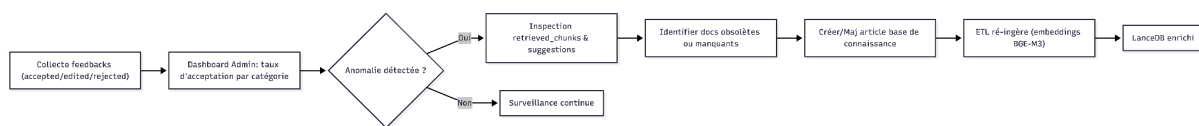
Justification des Champs :

1. `retrieved_chunks` : Ce champ est vital pour le débogage. Si l'IA répond mal, est-ce la faute du LLM (hallucination) ou du RAG (documents sources non pertinents récupérés)?.
2. `model_config_snapshot` : Permet de tracer quelle version du modèle a produit quelle erreur.
3. `final_content` : C'est la donnée "Or". Elle permet de constituer des paires (Question + Contexte) -> (Réponse Idéale) pour entraîner des modèles spécialisés futurs.

5.3. Exploitation des données

Ce système de feedback alimente deux processus vertueux :

1. **Monitoring de la Qualité (Dashboard Admin)** : Suivi du taux d'acceptation par catégorie de ticket. Une chute du taux sur la catégorie "Imprimantes" signale peut-être que la documentation source (Lot 2) sur les imprimantes est obsolète.
2. **Maintenance de la Base de Connaissance** : Si un solveur modifie substantiellement une réponse et que le ticket est clôturé avec succès, le système peut suggérer automatiquement aux administrateurs de créer un nouvel article de Knowledge Base basé sur cette "nouvelle" solution validée, bouclant ainsi la boucle avec le Lot 2.



6. Détails d'implémentation et sécurité

6.1. Structure du prompt système (template optimisé)

Le prompt est le "code source" de l'IA. Voici une structure de template Jinja2 optimisée pour le support technique en français, intégrant les meilleures pratiques (Rôle, Tâche, Contexte, Format).

Rôle: Tu es un assistant expert senior au sein du support technique de Ticketack. Ta mission est d'aider le solveur à résoudre le ticket efficacement.

Tâche: Analyse le problème décrit et propose une solution technique précise, étape par étape.

Contexte (Documents récupérés de la base de connaissance):
{% for doc in documents %}

```

<document id="{{ doc.id }}" source="{{ doc.source }}">
  {{ doc.content }}
</document>
{% endfor %}

Demande du Client:
Titre: {{ ticket.title }}
Description: {{ ticket.description }}
Métadonnées: {{ ticket.metadata }}

Instructions de Sécurité et Qualité:
1. Base-toi EXCLUSIVEMENT sur les documents fournis dans le contexte.
Si les documents ne contiennent pas la réponse, indique-le clairement
et propose des pistes de diagnostic générales.
2. Ne jamais inventer de commandes, de chemins de fichiers ou de
procédures qui ne sont pas dans le contexte.
3. Adopte un ton professionnel, technique mais pédagogique.

Format de Sortie Attendu (JSON Strict):
Réponds UNIQUEMENT avec un objet JSON respectant ce schéma :
{
  "summary": "Résumé du problème identifié en 1 phrase",
  "analysis": "Analyse technique de la cause probable",
  "steps": ["Étape 1", "Étape 2", ...],
  "missing_info": "Questions à poser au client si le ticket est
incomplet",
  "confidence_score": 0.0 à 1.0,
  "citations":
}

```

6.2. Sécurité des données et conformité

L'introduction de l'IA générative crée de nouveaux vecteurs de risques qu'il faut mitiger dès la conception.

1. **Anonymisation (Sanitization)** : Avant d'envoyer le contenu du ticket au LLM (même local), un pré-traitement doit être appliqué pour masquer les données personnelles (PII) détectables par Regex (emails, téléphones, IP, numéros de sécurité sociale). Bien que le modèle soit auto-hébergé, c'est une mesure de "Défense en Profondeur".
2. **Protection contre l'Injection de Prompt** : Les données utilisateur (le corps du ticket) sont traitées comme des données non fiables. Elles sont encapsulées dans des balises claires (ex: `<user_input>`) dans le prompt pour empêcher le modèle de confondre une instruction malveillante contenue dans le ticket avec les instructions système.

Cependant nous ne voyons pas de risque d'affichage d'informations sensibles, étant donné que nous avons fait le choix de ne laisser l'accès qu'aux Solveurs. Nous considérons que les Solveurs ont le droit d'accès à l'ensemble de la base de connaissance (choix fait à partir du lot 2), donc l'IA ne devrait pas créer le risque qu'une information sensible apparaisse pour le Solveur.

7. Glossaire

(1) **Intelligence artificielle d'assistance aux solveurs**

Composant du système Ticketack chargé d'analyser automatiquement le contenu des tickets et de générer des propositions de résolution adaptées afin d'aider les solveurs à traiter les demandes plus rapidement et plus efficacement.

(2) **RAG – Retrieval Augmented Generation**

Mécanisme combinant un modèle d'intelligence artificielle générative et une base de connaissances. Il permet d'enrichir les réponses de l'IA en s'appuyant sur des tickets référencés, des documents internes et des procédures existantes, garantissant ainsi des suggestions contextualisées et cohérentes.

(3) **Pistes de résolution contextualisées**

Suggestions de solutions générées par l'IA à partir de l'analyse sémantique du ticket (titre, description, pièces jointes) et des données issues de la base de connaissances. Elles peuvent inclure des actions à mener, des procédures à suivre ou des références à des tickets similaires.

(4) **Solveur**

Utilisateur disposant du rôle lui permettant de traiter et résoudre les tickets. Il peut consulter les suggestions de l'IA, les accepter ou les refuser, fournir des commentaires explicatifs et participer ainsi à l'amélioration continue du système.

(5) **Hallucination** Phénomène où l'IA génère une réponse qui semble très crédible et bien écrite, mais qui est factuellement fausse ou inventée de toutes pièces (par exemple, inventer une procédure qui n'existe pas).

(6) **Data Flywheel (Boucle d'apprentissage)** Cercle vertueux où chaque interaction du solveur (accepter, corriger ou refuser une solution) est enregistrée pour affiner et améliorer automatiquement la pertinence de l'IA au fil du temps.

(7) **Prompt** Ensemble des instructions scénarisées envoyées à l'IA (définition de son rôle d'expert, contexte documentaire, format de réponse attendu) pour qu'elle traite la demande du client selon les règles précises de l'entreprise.

(8) **Token** L'unité de mesure de base de l'IA (équivalent à environ 3/4 d'un mot) utilisée pour calculer la capacité de "mémoire" du modèle (combien de texte il peut lire d'un coup) et sa vitesse de traitement.

(9) **LLM (Large Language Model)** Le "moteur" d'intelligence artificielle générative capable de comprendre et produire du texte (ex: Mistral, Qwen). Dans ce projet, c'est un composant interchangeable qui peut être remplacé sans refaire toute l'application.

(10) **Inférence** L'action pour le serveur de "réfléchir" mathématiquement et de générer la réponse textuelle. C'est l'étape qui consomme de la puissance de calcul et prend du temps.

(11) **VRAM (Mémoire Vidéo)** La mémoire vive spécifique de la carte graphique du serveur. C'est la ressource matérielle critique qui détermine la taille (et donc l'intelligence) du modèle d'IA que l'on peut installer.

(12) **Embedding (Vectorisation)** Technique de traduction du texte (tickets, documents) en une suite de chiffres pour permettre au système de trouver des documents par le "sens" (thématique) plutôt que par mots-clés exacts.

- (13) **Quantification (4-bit)** Technique de compression du modèle d'IA permettant de réduire sa taille pour le faire fonctionner sur des serveurs standards moins coûteux, avec une perte de qualité quasi imperceptible.
- (14) **Orchestrateur IA** Le programme informatique "chef d'orchestre" (développé en Python) qui fait le lien entre l'application Ticketack et le moteur d'IA, assurant que les demandes ne soient jamais perdues même si le système est chargé.
- (15) **Sanitization (Anonymisation)** Processus de sécurité automatique qui nettoie les données du ticket (masquage des emails, téléphones, données personnelles) *avant* de les envoyer à l'IA pour analyse.